Agent Remediation Instructions

RoxCrew / MyShift - security hardening and Roxi/Roxer deployment readiness

Prepared: October 27, 2026
Purpose: give the implementation agent an exact, evidence-based remediation plan. This document supersedes any security summary that claims all controls are already complete.

NON-NEGOTIABLE DEPLOYMENT GATE

Do not deploy this application to public production or expose real user data until every BLOCKER task is complete and the acceptance tests pass. You may use an isolated staging environment only after removing the authentication bypasses and demo credentials.

Working rules for the agent

- Work in the current repository and preserve unrelated user changes.
- Before editing, inspect the current implementation; do not assume the previous executive summary is accurate.
- Do not add test-account bypasses, header-based impersonation, fallback secrets or production demo seeds.
- Use environment variables or a managed secret store for all secrets.
- After each phase, run the relevant tests and report files changed, commands run and remaining risks.
- Do not deploy, publish a domain or migrate production data without explicit authorization.

Current assessment

Area	Current status	Decision
Authentication	**BLOCKER**	Not production safe: header impersonation and admin fallback remain.
Session storage	**HIGH**	Cookie exists, but JWT is also stored in browser storage.
Tenant isolation	**HIGH**	Creation is checked; invitation listing/revocation need stronger board/tenant checks.
Card integrity	**PASS**	moveCard checks target list board ownership.
Uploads	**HIGH**	Access checks exist, but storage is local and limits differ.
Build/deploy	**TODO**	No clean build proof; start uses tsx from devDependencies.

Phase P0 - Remove authentication bypasses

Target files: server/middleware.ts, server/auth.ts, server/routes/authRoutes.ts, src/lib/api.ts, server/db.ts.

P0.1 Delete impersonation fallbacks

- Remove x-user-email and x-user-id authentication from requireAuth.
- Remove the automatic admin@grandregency.com fallback.
- Keep only a valid HttpOnly session cookie or a deliberately supported Bearer token.
- If Bearer tokens are not required by a real client, remove that path too.

P0.2 Remove test credentials from production

- Delete admin@test.com, admin, Password123! and any equivalent login bypass from authRoutes.ts and db.ts.
- Separate demo fixtures from production startup; never call ensureAdminTestUser in production.
- Rotate any credentials that may have been exposed in the repository or documentation.

P0.3 Enforce secret policy

- Remove the development fallback hospitality_kanban_secure_jwt_secret_key_2026_dev.
- Fail startup unless JWT_SECRET exists and is generated with high entropy; a length check alone is not enough.
- Use a key rotation plan and include a token version or session revocation mechanism if operationally required.

P0.4 Fix browser token handling

- Stop returning the JWT in signup/login JSON unless an approved non-browser client requires it.
- Remove localStorage and sessionStorage token persistence from src/lib/api.ts.
- Use the HttpOnly cookie as the browser session.

P0.5 Fix CSRF and cookie policy

- Use SameSite=Lax or Strict for same-origin deployment.
- If cross-site cookies are required, add CSRF tokens and strict Origin/Referer validation.
- Keep Secure and HttpOnly enabled in production.

P0 acceptance tests

```
Expected results: unauthenticated requests return 401; fake x-user-email/x-user-id headers do not
authenticate; admin@test.com with known demo passwords fails; a valid session works; cross-site
state-changing requests fail.
```

Phase P0 - Tenant, board and file authorization

Target files: server/routes/invitationRoutes.ts, server/routes/boardRoutes.ts, server/middleware.ts, server.ts.

P0.6 Centralize authorization

- Create one authorization helper that verifies user identity, board existence, board membership/role and tenant ownership.

- Use it for invitation creation, invitation listing, invitation revocation, board mutations, card mutations and file access.

- Do not authorize Managers only by global role when the request targets a specific board.

P0.7 Invitation isolation

- Require the target board to belong to the current user's tenant for POST /api/invitations.

- Apply the same check to GET /api/invitations/board/:boardId and DELETE /api/invitations/:id.

- Do not return invitation tokens to users who are not authorized for that exact board.

P0.8 Card movement and deletion

- Preserve the existing moveCard check that targetList.board_id equals card.board_id.

- Add a test proving cross-board movement is rejected without changing data.

- For image deletion, accept only an attachment already recorded on the authorized card; keep path.basename and restrict the final path to the storage root.

P0.9 File access model

- Do not grant every Manager access to every board wallpaper across tenants.

- Do not grant global Admin access to files outside the intended tenant unless this is an explicit policy.

- Prefer private object storage and short-lived signed URLs over local public paths.

P0 acceptance tests

```
Expected results: a Manager from tenant A cannot list, revoke, view or delete tenant B invitations/files;
a Viewer cannot mutate cards; a card cannot move to a list on another board; traversal and arbitrary
client paths are rejected.
```

Phase P1 - Abuse controls, uploads and API validation

Target files: server.ts, server/routes/authRoutes.ts, server/routes/boardRoutes.ts, server/routes/aiRoutes.ts.

P1.1 Rate limiting

- Keep the 15 requests per 15 minutes authentication limiter, but add account-aware throttling or progressive lockout.

- Keep the general API limiter, but use a shared store if the deployment has multiple instances.

- Add dedicated limits for invitations, AI card creation, uploads and password reset flows.

- Verify trust proxy matches Roxer's actual proxy chain; do not blindly trust spoofable client IP headers.

P1.2 Enumeration and validation

- Use generic responses for duplicate signup emails if account enumeration is not an intended feature.

- Validate all JSON with a schema library and reject unexpected fields.

- Set smaller body limits per route; 50 MB JSON bodies are excessive for normal API calls.

- Validate dates, IDs, roles, positions, max_uses and expires_in_hours with bounded ranges.

P1.3 Upload controls

- Document and enforce one consistent maximum or separate limits explicitly: current code is 5 MB for avatars, 8 MB for card images and 1 MB for wallpapers.

- Validate file signatures/magic bytes, not only client MIME types.

- Reject executable SVG/HTML content unless it is sanitized and served with safe content disposition.

- Generate server-side filenames, enforce storage quotas, clean orphaned files and consider malware scanning.

P1.4 AI endpoint

- Remove the fallback roxcrew_claude_secret_2026.

- Require authentication or a separately scoped webhook key.

- Authorize the target board and list, validate all fields, rate limit the endpoint and avoid accepting secrets in request bodies when headers are available.

P1 acceptance tests

```
Expected results: brute-force attempts are throttled; oversized bodies/files are rejected; disguised files are rejected; malformed IDs and roles return 400; unauthorized AI requests return 401/403; no response reveals whether an email exists.
```

Phase P1 - Runtime, persistence and Roxi/Roxer adaptation

Target files: package.json, server.ts, server/db.ts, upload code, deployment configuration.

P1.5 Production build

- Pin the supported Node version, preferably Node 20 if required by Roxer.

- Compile the backend TypeScript to JavaScript and use node dist-server/server.js in production, or explicitly guarantee tsx is installed at runtime.

- Make build validate both frontend and backend. Do not rely only on vite build.

- Add a clean-install lockfile strategy and run npm ci or the approved Bun equivalent in CI.

P1.6 Persistent data

- Replace in-memory Maps plus data/database.json with SQLite at /mnt/data only after confirming Roxer's persistence semantics, or use managed PostgreSQL.

- Create migrations, constraints, transactions and backup/restore procedures.

- Remove data/database.json and demo user seeds from production packages.

- Test restart, redeploy, cold start, concurrent writes and multiple instances.

P1.7 Upload storage

- Move avatars, card photos and wallpapers to private durable object storage.

- Store object keys, not public filesystem URLs, in the database.

- Return signed download URLs only after authorization.

- Define retention and deletion behavior.

P1.8 Real-time behavior

- Confirm official Roxer support for Socket.IO/WebSocket connections, polling upgrades, timeouts and reconnection.

- If unsupported, replace the in-memory presence/event model with a supported polling/SSE/pub-sub architecture.

- Test events across separate instances; do not assume activeBoardUsers is shared.

P1 acceptance tests

```
Expected results: clean build/start succeeds on a fresh environment; data survives restart/redeploy;
uploads survive redeploy; two instances see the same data; real-time events either work as documented or
the alternative architecture passes.
```

Phase P2 - Headers, testing and release gate

P2.1 Security headers

- Keep Helmet enabled in production.

- Remove unsafe-eval from the production CSP if the frontend permits it; eliminate unsafe-inline where practical using nonces or hashes.

- Restrict connectSrc to the official HTTPS and WSS origins.

- Use HSTS preload only after every relevant domain/subdomain is permanently HTTPS.

- Confirm frame, referrer, permissions and cross-origin policies for the final architecture.

P2.2 Automated test suite

- Add unit tests for JWT verification, password handling, role hierarchy and tenant comparison.

- Add integration tests for login, signup, invitation lifecycle, card movement, file access and AI endpoint.

- Add negative tests for missing/invalid JWT, impersonation headers, cross-tenant IDs, path traversal, oversized files and malformed JSON.

- Add a clean install, typecheck, build, dependency audit and smoke-test job in CI.

P2.3 Required evidence before deployment

Evidence	Required result	Status
Authentication tests	No header impersonation; demo credentials fail; cookie session works.	**TODO**
Tenant tests	Tenant A cannot access tenant B boards, invites or files.	**TODO**
Build proof	Fresh environment passes typecheck, frontend build and backend build.	**TODO**
Persistence proof	Restart/redeploy and multi-instance tests pass.	**TODO**
Dependency audit	Lockfile-backed audit has no unresolved high/critical issue.	**TODO**
Staging smoke test	Login, board, invite, upload, permissions and real-time flows pass.	**TODO**
Rollback/backup	Backup restoration and rollback are tested.	**TODO**

Release decision

The agent must report PASS or BLOCKED for each evidence item. Production deployment is BLOCKED if any authentication bypass, default credential, tenant isolation failure, data persistence failure or unresolved critical dependency vulnerability remains.

Final instruction to the agent

Implement the phases in order. Do not claim a control is complete based on comments or configuration alone. Prove each control with code review and a test. At the end, provide a concise change log, test output, deployment assumptions, remaining risks and explicit confirmation that production deployment is or is not approved.

Evidence reviewed

Current repository files include server/auth.ts, server/middleware.ts, server.ts, server/routes/authRoutes.ts, server/routes/invitationRoutes.ts, server/routes/boardRoutes.ts, server/routes/aiRoutes.ts, server/db.ts, server/socket.ts, src/lib/api.ts and package.json. Platform assumptions are based on https://roxer.com/docs/.